

qcBot

Quick-Start Guide

Playwright & multi-stack test quality analyser — get running in under 5 minutes



Node.js
CLI



Python
SDK



Browser
UI



GitHub
Actions



AI auto-
fix



Contents

1

[Two rule engines](#)

3

[Python CLI & SDK](#)

5

[GitHub Actions](#)

7

[Complete rules reference \(59 rules\)](#)

9

[Troubleshooting](#)

2

[Node.js CLI](#)

4

[Browser UI](#)

6

[Agentic AI — all 4 agents](#)

8

[Supported stacks](#)

1 TWO RULE ENGINES — HOW THEY WORK TOGETHER

ENGINE 1 Predefined rules

59 built-in regex rules that scan your code line-by-line. **Instant, offline, no API key needed.** Every rule has a fixed ID, severity, and fix guidance.

CLI COMMANDS

```
qcbot check ./tests
```

Run all 59 rules — prints findings summary to terminal

```
qcbot check ./tests --report --threshold 80
```

Same + generate HTML report + exit code 1 if score < 80

```
qcbot check ./tests --stack  
pytest_api --json result.json
```

Different stack + write raw JSON output to file

```
qcbot check ./tests --disabled-rules  
PW-NMC-010,PW-CPX-003
```

Skip specific rules for this run

BROWSER UI

- Load folder → Findings tab — all violations appear instantly
- Toggle rules on/off in the **Rules tab** (per-stack, saved across sessions)
- No setup required — works out of the box

ENGINE 2 AI rules (agentic)

AI reads your actual code and provides **contextual explanations + corrected code**. Also discovers **new custom rules** specific to your codebase. Requires an API key (Anthropic / Google / AWS Bedrock).

CLI COMMANDS

```
qcbot fix --file  
./tests/login.spec.ts --rule PW-REL-001
```

Agent 3 — AI explains one finding + shows corrected code

```
qcbot remediate ./tests --severity  
critical --dry-run
```

Agent 1 — AI fixes all critical findings (dry-run = show diff only)

```
qcbot remediate ./tests --severity  
critical --commit --branch fix/auto
```

Agent 1 — same but writes to disk and git-commits on a new branch

```
qcbot suggest-rules ./tests --  
provider anthropic
```

Agent 4 — AI reads your code and proposes up to 6 new custom rules

```
qcbot pr-review --repo owner/repo --  
pr 42 --token ghp_...
```

Agent 2 — post inline findings as GitHub PR review comments

BROWSER UI

- ⚙ Settings → paste API key → Save
- Findings: **Get AI fix** on any finding · ✨ **Fix all critical** for batch
- Rules: ✨ **Suggest rules** to discover new ones from your codebase

Both engines work together. Engine 1 always runs first — instant scan, no key needed. Engine 2 is on-demand — triggered by you to explain, fix, or discover rules. Use Engine 1 alone if you just want the findings list. Add Engine 2 when you need AI-assisted fixes or custom rule discovery.

Works for all languages — just change `--stack`

The same CLI commands apply to every language and framework. The `--stack` flag tells qcBot which rule set to load for your codebase. Run `qcbot stacks` to see all options.

Language / Framework	Stack value	CLI command (Engine 1)	AI fix command (Engine 2)
Playwright TypeScript	<code>playwright</code>	<code>qcbot check ./tests --stack playwright</code>	<code>qcbot remediate ./tests --stack playwright --severity critical</code>
Playwright Python	<code>playwright_python</code>	<code>qcbot check ./tests --stack playwright_python</code>	<code>qcbot remediate ./tests --stack playwright_python --severity critical</code>
Python / pytest	<code>pytest_api</code>	<code>qcbot check ./tests --stack pytest_api</code>	<code>qcbot remediate ./tests --stack pytest_api --severity critical</code>
Java REST Assured	<code>restassured</code>	<code>qcbot check ./src/test --stack restassured</code>	<code>qcbot remediate ./src/test --stack restassured --severity critical</code>
Java API (generic)	<code>java_api</code>	<code>qcbot check ./src/test --stack java_api</code>	<code>qcbot remediate ./src/test --stack java_api --severity critical</code>
Karate DSL	<code>karate</code>	<code>qcbot check ./src/test --stack karate</code>	<code>qcbot remediate ./src/test --stack karate --severity critical</code>
Postman / Newman	<code>postman</code>	<code>qcbot check ./collections --stack postman</code>	<code>qcbot remediate ./collections --stack postman --severity critical</code>
TypeScript (generic)	<code>typescript</code>	<code>qcbot check ./src --stack typescript</code>	<code>qcbot remediate ./src --stack typescript --severity critical</code>

Python SDK users — same logic, Python syntax

```
qcbot-py check ./tests --stack pytest_api --threshold 80 --report
```

```
qcbot-py check ./tests --stack playwright_python --json | python3 -c "import
```

```
sys,json; r=json.load(sys.stdin); print(r['score'])"
```

2 NODE.JS CLI — QUICKEST PATH

Prerequisites: Node.js 18 or higher — download at nodejs.org/en/download (choose the LTS version). No other installs required.

1 Install the CLI globally (one-time only)

This adds `qcbot` as a command you can run anywhere on your machine.

```
npm install -g @cqs/qcbot
```

→ Or skip the install and use npx for a one-shot run: `npx @cqs/qcbot check ./tests`

2 Run a quality check on your test folder

Point it at the directory that contains your spec / test files. Replace `./functional-tests` with your actual path.

```
qcbot check ./functional-tests
```

→ Defaults: stack = playwright, threshold = 80, 4 parallel workers

3 Generate an HTML report (recommended)

Adds a self-contained report you can open in any browser and share with the team.

```
qcbot check ./functional-tests --report
```

→ Report saved to: `./qcbot-report/report.html` — open it with `open qcbot-report/report.html`

4 What you'll see in the terminal

```
⚡ Analysing 168 files • stack: playwright • threshold: 80
```

```
Score  72 Fair
```

```
Files 168
```

```
Critical 3 Warnings 47 Total 284
```

```
Status ✖ FAILED (threshold 80)
```

```
CRITICAL Hard-coded wait • login.spec.ts:42
```

```
CRITICAL Missing assertion • dashboard.spec.ts:18
```

```
WARNING Cyclomatic complexity 14 > 12 • utils.ts:33
```

```
✓ HTML report → ./qcbot-report/report.html
```

All available flags

Flag	Default	What it does
<code>--stack <id></code>	<code>playwright</code>	Rule set to use. Run <code>qcbot stacks</code> to see all options.
<code>--threshold <n></code>	<code>80</code>	Minimum passing score (0–100). Exit code 1 if score is below this.
<code>--report</code>	off	Write a self-contained HTML report to <code>./qcbot-report/</code> .
<code>--output <dir></code>	—	Override the output directory for the HTML/JSON report.
<code>--parallel <n></code>	<code>4</code>	Number of files to analyse simultaneously.
<code>--json <file></code>	—	Write raw JSON result to a file (useful for CI or scripting).
<code>--no-cross-file</code>	off	Skip cross-file duplicate test detection.
<code>--project <name></code>	—	Project name shown in the HTML report header.

5

Set your AI API key (one-time per machine)Only needed for AI agents — `qcbot fix`, `qcbot remediate`, `qcbot suggest-rules`

» Skip if you only need Engine 1 (predefined rules)

Get a free API key from your chosen provider, then set it as an environment variable once. qcBot reads it automatically every time — you never need to enter it again.

Provider	Get key at	Env variable to set
Anthropic Claude 🌟	console.anthropic.com → API Keys	<code>ANTHROPIC_API_KEY</code>
Google Gemini	aistudio.google.com → Get API Key	<code>GOOGLE_API_KEY</code>
AWS Bedrock	AWS Console → IAM → access key	<code>AWS_ACCESS_KEY_ID</code> + <code>AWS_SECRET_ACCESS_KEY</code>

Mac / Linux — paste in terminal, then add to your shell profile:

```
export ANTHROPIC_API_KEY=sk-ant-api03-YOUR_KEY_HERE
# Make it permanent (runs automatically on every new terminal):
echo 'export ANTHROPIC_API_KEY=sk-ant-api03-...' >> ~/.zshrc && source ~/.zshrc
```

Windows (PowerShell) — set for current session + persist:

```
$env:ANTHROPIC_API_KEY = "sk-ant-api03-YOUR_KEY_HERE"
# Persist across sessions:
[System.Environment]::SetEnvironmentVariable("ANTHROPIC_API_KEY", "sk-ant-...",
"User")
```

Verify it's set — run this in a new terminal:

```
echo $ANTHROPIC_API_KEY # should print your key, not blank
```

✓ **Done — that's it.** Every agentic command (`qcbot fix` , `qcbot remediate` , `qcbot suggest-rules`) will pick up the key automatically from your environment. You never need to set it again on this machine. To use a different key for one command: add `--api-key YOUR_KEY --provider anthropic` .

Useful extra commands:

`qcbot stacks` — list all supported stacks (playwright, typescript, pytest_api, java_api, karate, postman, and more)

`qcbot remediate ./tests --severity critical --dry-run` — AI agent that auto-fixes critical findings (dry-run shows changes without writing)

3 PYTHON CLI & SDK

Prerequisites: Python 3.9+ and Node.js 18+. The Python package shells out to Node internally — the rule engine always runs in Node.js.

1 Install the Python package

The `[cli]` extra installs Click so the `qcbot-py` command is available in your terminal.

```
pip install "qcbot[cli]"
```

→ Library-only (no terminal command): `pip install qcbot`

2 Run from the terminal

Same flags as the Node CLI — just the command name changes.

```
qcbot-py check ./functional-tests --threshold 80 --report
```

```
qcbot-py stacks
```

3 Use as a Python library

Import directly in any Python script, pytest hook, Django/Flask view, or Jupyter notebook.

```
from pw_quality import check

result = check("./functional-tests", threshold=80, report=True)

print(result.score)           # 72
print(result.passed)          # False
print(result.grade)           # "Fair"
print(result.summary.critical) # 3

# Per-file breakdown
for file in result.files:
    print(f"{file.score:3} {file.name} ({file.critical_count} critical)")

# All critical findings
for finding in result.critical_findings:
    print(f"{finding.title} - {finding.file}:{finding.line}")
```

4 Add a quality gate to pytest (conftest.py)

```
from pw_quality import check, PwQualityError

def pytest_sessionfinish(session, exitstatus):
```

```
try:
    result = check("./tests", threshold=80)
    if not result.passed:
        print(f"\n△ Quality gate FAILED – score {result.score} < 80")
except PwQualityError as e:
    print(f"\n△ qcbot unavailable: {e}")
```

4 BROWSER UI (VISUAL DASHBOARD)

1 Start the dev server

From inside the project folder, install dependencies and start. The app opens at

`http://localhost:5173`.

```
cd playwright-quality-studio
```

```
npm install && npm run dev
```

2 Load your test files

On the welcome screen click **Load folder** and pick the directory containing your spec files (e.g. `functional-tests`). All files are scanned automatically — no configuration needed.

You can also drag and drop a folder directly onto the welcome screen.

3 Navigate the results



Findings tab

All rule violations, filterable by severity. Click any finding to see the rule detail and get an AI-powered fix suggestion.



Rules tab

Toggle any rule on or off per stack. Use ✨ **Suggest rules** to let AI analyse your codebase and propose new rules.



Duplicates tab

Cross-file duplicate test detection with similarity scores — spot copy-pasted tests instantly.



Summary tab

Score breakdown by category (selectors, assertions, reliability, etc.) with charts and per-file rankings.

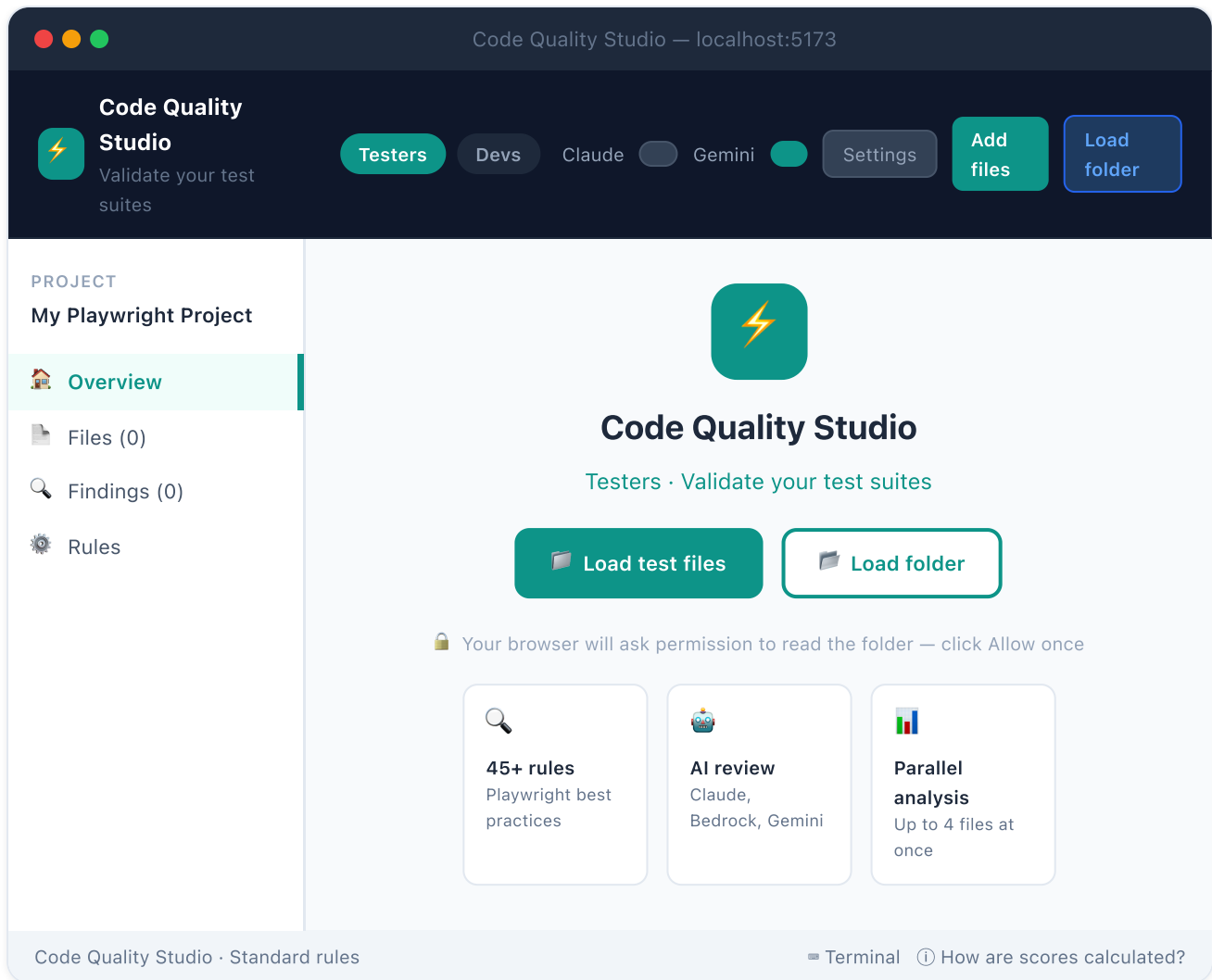
4 Enable AI features (optional)

Click the **Settings** button in the top-right header → check **Use AI for new scans** → paste your API key → Done. Two new buttons appear:

- ✨ **Fix all critical (N)** — in the Findings tab header. Calls AI on every critical finding in batch and shows corrected code you can copy.
- ✨ **Suggest rules from codebase** — in the Rules tab header. AI reads up to 8 of your files and proposes custom rules with patterns and fixes. Add any in one click.

→ Supported providers: Anthropic (Claude), Google Gemini, AWS Bedrock

Browser UI — what each screen looks like



The screenshot shows the 'Code Quality Studio' application running on 'localhost:5173'. The header bar is dark blue with the title 'Code Quality Studio' and a subtitle 'Validate your test suites'. Below the header, there's a navigation bar with tabs: 'Testers' (active), 'Devs', 'Claude', 'Gemini', 'Settings', 'Add files', and 'Load folder'. The main content area is divided into a left sidebar and a main panel. The sidebar, titled 'PROJECT My Playwright Project', has a list of items: 'Overview' (selected), 'Files (0)', 'Findings (0)', and 'Rules'. The main panel features a large green lightning bolt icon, the title 'Code Quality Studio', and the subtitle 'Testers · Validate your test suites'. Below this are two buttons: 'Load test files' and 'Load folder'. A message states: 'Your browser will ask permission to read the folder — click Allow once'. At the bottom of the main panel, there are three cards: '45+ rules Playwright best practices', 'AI review Claude, Bedrock, Gemini', and 'Parallel analysis Up to 4 files at once'. The footer bar shows 'Code Quality Studio · Standard rules' on the left and 'Terminal' and 'How are scores calculated?' on the right.

Code Quality Studio — localhost:5173

Code Quality Studio
Validate your test suites

Testers Devs Claude Gemini Settings Add files Load folder

PROJECT
My Playwright Project

Overview Files (0) Findings (0) Rules

Code Quality Studio
Testers · Validate your test suites

Load test files Load folder

🔒 Your browser will ask permission to read the folder — click Allow once

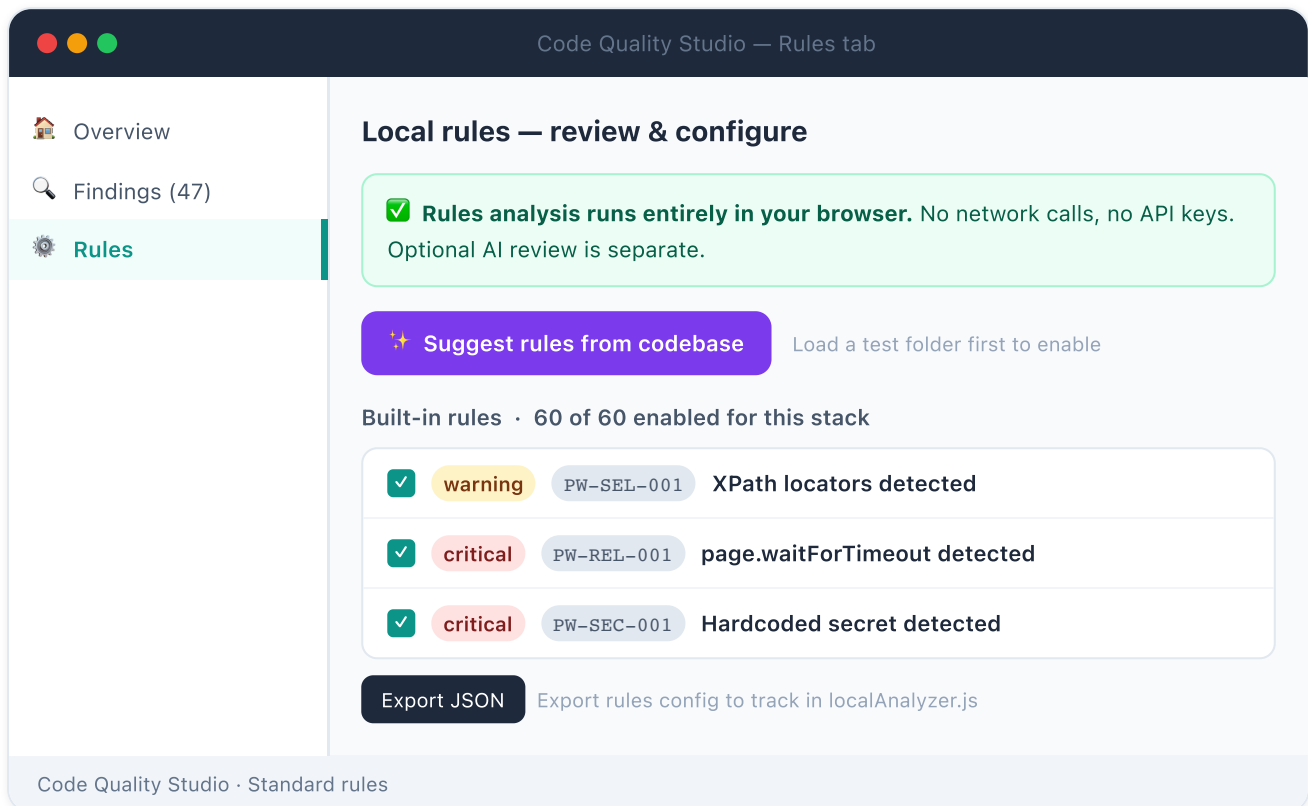
45+ rules
Playwright best practices

AI review
Claude, Bedrock, Gemini

Parallel analysis
Up to 4 files at once

Code Quality Studio · Standard rules Terminal ⓘ How are scores calculated?

① Welcome / Overview screen — click **Load folder** to pick your test directory



The screenshot shows the 'Code Quality Studio' application on the 'Rules tab'. The header bar is dark blue with the title 'Code Quality Studio' and subtitle 'Rules tab'. The left sidebar is titled 'Overview' and has a list of items: 'Overview', 'Findings (47)', and 'Rules' (selected). The main panel is titled 'Local rules — review & configure'. It features a green box with a checkmark and the text: 'Rules analysis runs entirely in your browser. No network calls, no API keys. Optional AI review is separate.' Below this is a purple button 'Suggest rules from codebase' with a subtext 'Load a test folder first to enable'. A section titled 'Built-in rules · 60 of 60 enabled for this stack' contains a list of rules: 'warning PW-SEL-001 XPath locators detected', 'critical PW-REL-001 page.waitForTimeout detected', and 'critical PW-SEC-001 Hardcoded secret detected'. At the bottom, there's a button 'Export JSON' with the text 'Export rules config to track in localAnalyzer.js'. The footer bar shows 'Code Quality Studio · Standard rules'.

Code Quality Studio — Rules tab

Overview Findings (47) Rules

Local rules — review & configure

✅ Rules analysis runs entirely in your browser. No network calls, no API keys. Optional AI review is separate.

🌟 Suggest rules from codebase Load a test folder first to enable

Built-in rules · 60 of 60 enabled for this stack

✅ warning PW-SEL-001 XPath locators detected

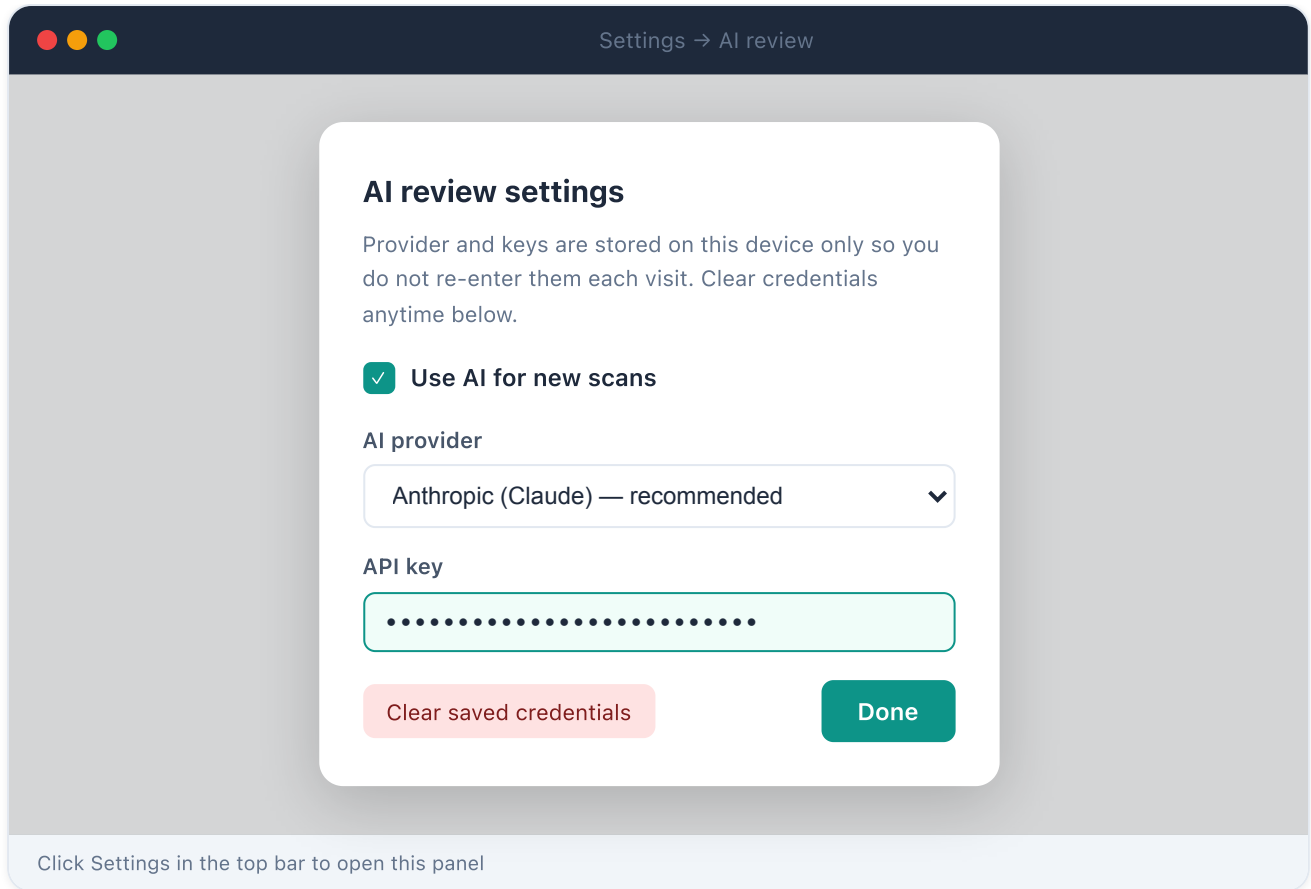
✅ critical PW-REL-001 page.waitForTimeout detected

✅ critical PW-SEC-001 Hardcoded secret detected

Export JSON Export rules config to track in localAnalyzer.js

Code Quality Studio · Standard rules

② Rules tab — toggle rules on/off · 🌟 Suggest rules from codebase discovers custom rules via AI



③ Settings panel — paste your API key here. Stored locally, never sent to our servers.

5 GITHUB ACTIONS — AUTOMATIC PR QUALITY GATE

Create the file `.github/workflows/quality-pr.yml` in your repository. The workflow runs on every pull request automatically — no secrets needed, GitHub provides the token.

```
name: qcbot gate

on:
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  contents: read
  pull-requests: write # needed for inline PR comments

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: PR quality review
        run: npx --yes @cqs/qcbot pr-review \
          --stack playwright --threshold 80
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

      - name: Full quality check
        run: npx @cqs/qcbot check ./tests --report --threshold 80
        continue-on-error: true

      - name: Upload HTML report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: quality-report
          path: qcbot-report/report.html
          retention-days: 30
```

✓ **Pass** Score $\geq$ threshold → workflow approves the PR automatically

✗ **Fail** Score $<$ threshold → inline comments posted on each finding line in the PR diff

📄 **Report** HTML report always uploaded as a build artifact (available for 30 days)

6 AGENTIC AI FEATURES — ALL 4 CLI COMMANDS

What is agentic AI? These commands go beyond static analysis — they call an AI model (Anthropic Claude, Google Gemini, or AWS Bedrock) in a loop to understand your code, explain findings, and write fixes. You need an API key for features 1–3. Feature 4 (PR review) uses GitHub's own token.

🔑 **API key setup is a one-time step.** See [Section 2 → Step 5](#) for the full walkthrough (provider table, Mac/Linux & Windows commands, verify step). Once set in your shell profile it works automatically for every command below — you never enter it again.

» Skip if you only use Engine 1 — `qcbot check` runs the 59 predefined rules with no key at all.

AGENT 1 ✨ Remediation agent — bulk auto-fix

Scans all your test files, picks every finding that matches the severity filter, calls AI to generate a corrected version of each file, and writes the fixes to disk. Optionally creates a git branch and commit so it's easy to review as a PR.

a Dry run — see what AI would change, write nothing

```
qcbot remediate ./functional-tests --severity critical --dry-run
```

→ Prints a before/after diff for each file without touching anything on disk

b Apply fixes and auto-commit to a new branch

```
qcbot remediate ./functional-tests --severity critical --commit --branch  
fix/qcbot-auto
```

→ Set `ANTHROPIC_API_KEY=sk-ant-...` in your environment, or pass `--api-key YOUR_KEY`

c Fix warnings too, using Google Gemini

```
qcbot remediate ./functional-tests --severity all --provider google --ap  
i-key YOUR_GOOGLE_KEY
```

🖥️ WHAT YOU SEE IN THE TERMINAL

```
$ qcbot remediate ./functional-tests --severity critical --dry-run
⚡ qcBot remediation agent • provider: anthropic • severity: critical

Scanning ./functional-tests ...
Found 3 files with critical findings (12 total)

— login.spec.ts —
```

```
- await page.waitForTimeout(2000);
+ await page.waitForSelector('#submit-btn');

- await page.locator('//button[@id="login"]').click();
+ await page.getByRole('button', { name: 'Login' }).click();

— checkout.spec.ts —
- const password = 'SuperSecret123';
+ const password = process.env.TEST_PASSWORD ?? '';
```

Dry run complete · 3 files · 12 fixes · nothing written to disk

Remove --dry-run to apply, or add --commit --branch fix/qcbot-auto to commit

Flag	Default	Description
<code>--provider</code>	<code>anthropic</code>	<code>anthropic</code> <code>google</code> <code>bedrock</code>
<code>--api-key</code>	env var	API key (or set <code>ANTHROPIC_API_KEY</code> / <code>GOOGLE_API_KEY</code>)
<code>--model</code>	auto	Override the model name (e.g. <code>claude-sonnet-5</code>)
<code>--severity</code>	<code>critical</code>	<code>critical</code> <code>warning</code> <code>all</code>
<code>--dry-run</code>	off	Show diff without writing any files
<code>--commit</code>	off	git commit after writing fixes
<code>--branch</code>	—	Create a new branch before committing

⚠ Safety rules — read before running without `--dry-run`

1

Only run against test files — enforced by `--test-files-only` ON by default

The agent has a built-in safety flag `--test-files-only` that is **enabled by default**. It checks every candidate file against recognised test-naming patterns (`*.spec.ts`, `*.test.ts`, `test_*.py`, `*Test.java`, `*.feature` ...) and silently skips anything that doesn't match — even if you accidentally point the command at a mixed directory.

```
# --test-files-only is ON by default — no extra flag needed
qcbot remediate ./my-project --severity critical --dry-run
# → skips src/utils.ts, src/components/Login.tsx etc. automatically
# → only processes login.spec.ts, checkout.test.ts, test_api.py ...

# To disable (only if you are certain the directory contains test files
only):
qcbot remediate ./functional-tests --severity critical --no-test-files-
```

`only``△ --no-test-files-only active — AI may rewrite non-test files.`**BROWSER UI — SETTINGS → "RESTRICT AI FIXES TO TEST FILES ONLY"****Restrict AI fixes to test files only** **ON — recommended**

✦ Fix all critical only processes `*.spec.ts`, `*.test.ts`, `test_*.py`, `*Test.java` ... Production source files are skipped automatically.

**Restrict AI fixes to test files only** **OFF — use with caution**

⚠ AI may rewrite non-test files. Only disable if you are certain the target folder contains test files only.

Open **Settings** in the top bar → the toggle is always visible, independent of AI being on or off. The setting persists in browser localStorage across sessions.

✅ **Safe by default.** CLI requires explicit `--no-test-files-only` and prints a yellow warning when active. The browser toggle shows a yellow caution badge when off.

2 Always dry-run first — read the diff before applying

Run with `--dry-run` first. The terminal prints a before/after diff for every file it would change. Only apply if the changes look correct.

```
# Step 1 — inspect what AI wants to change (writes nothing)
qcbot remediate ./functional-tests --severity critical --dry-run

# Step 2 — if diff looks good, apply on an isolated branch
qcbot remediate ./functional-tests --severity critical --commit --branch
fix/qcbot-auto
```

3 Human must review and test before merging — never auto-merge AI fixes

The agent commits to a new branch (`fix/qcbot-auto`). Raise it as a PR, run your full test suite against it, and have a team member review the diff before merging to `main`. If tests fail on the branch — **do not merge**.

Recommended workflow:

1. `qcbot remediate ./tests --dry-run` → review diff
2. `qcbot remediate ./tests --commit --branch fix/qcbot-auto` → AI applies + commits
3. Open PR: `fix/qcbot-auto` → `main`
4. CI runs your test suite on the branch → must pass ✅
5. Team member reviews the diff → approves
6. Merge only after step 4 + 5 pass

4 Rollback — how to undo AI fixes if tests fail

If tests fail on the AI-fixed branch, rollback is instant because the fixes are isolated to a git branch.

```
# Option A – delete the branch entirely (safest)
git branch -D fix/qcbot-auto
# main is untouched – nothing to undo there

# Option B – undo changes on disk without a branch (if you skipped --branch)
git checkout -- . # revert all AI-written files
git checkout -- tests/login.spec.ts # or revert one specific file

# Option C – already committed without a branch? revert the commit
git revert HEAD
```

This is why `--commit --branch` is the recommended default. It always leaves your main branch clean — rollback is just deleting a branch.

AGENT 2 ✨ PR review bot — inline GitHub comments

Fetches the changed files in a pull request, runs qcBot on them, and posts inline comments on each finding — directly inside the GitHub PR diff view. Used automatically by the GitHub Actions workflow (Section 5) but you can also trigger it manually.

a Run manually against a specific PR

```
qcbot pr-review --repo owner/my-repo --pr 42 --token ghp_YOUR_TOKEN
```

→ Gets all changed files in PR #42, analyses them, posts inline review comments

b Dry run — see what comments would be posted

```
qcbot pr-review --repo owner/my-repo --pr 42 --token ghp_YOUR_TOKEN --dry-run
```

c In GitHub Actions (auto-detected, no flags needed)

```
npx @cqs/qcbot pr-review --stack playwright --threshold 80
```

GitHub
→ Actions sets `GITHUB_TOKEN`, `GITHUB_REPOSITORY` and `GITHUB_REF` automatically — no manual flags required

🖥️ WHAT YOU SEE IN THE TERMINAL

```
$ qcbot pr-review --repo acme/my-app --pr 42 --token ghp_YOUR_TOKEN
⚡ qcBot PR review · repo: acme/my-app · PR #42
```

```

Fetching changed files ...
Found 4 changed spec files in PR #42

Analysing login.spec.ts ... checkout.spec.ts ... api.spec.ts ...

— Findings —
CRITICAL login.spec.ts:14 page.waitForTimeout (PW-REL-001)
CRITICAL login.spec.ts:27 XPath locator (PW-SEL-001)
WARNING checkout.spec.ts:8 networkidle wait (PW-REL-004)

Posting 3 inline review comments to GitHub ...
Posting summary comment ...

✓ PR review posted • Score: 61 • Grade: Fair • REQUEST CHANGES
View: https://github.com/acme/my-app/pull/42

```

What gets posted: A summary table at the top of the PR with the overall score + grade, then an inline comment on each finding line (up to 30 — GitHub's limit). Result: **APPROVE** if score $\geq$ threshold, **REQUEST CHANGES** if below.

AGENT 3 ✨ Per-finding AI fix — single issue explanation

For a single finding, asks AI to explain why it's a problem and produce a corrected code snippet. Useful for understanding a specific rule before applying bulk fixes.

a Get AI explanation for one finding

```
qcbot fix --file ./functional-tests/login.spec.ts --rule PW-REL-001
```

→ Prints an explanation and corrected code block for that specific rule in that file

b Or use the browser UI

In the Findings tab, click any finding → "Get AI fix" button → AI explanation + corrected code appears in the panel. Copy the fix with one click.

🖥️ WHAT YOU SEE IN THE TERMINAL

```

$ qcbot fix --file ./functional-tests/login.spec.ts --rule PW-REL-001
⚡ qcBot AI fix • file: login.spec.ts • rule: PW-REL-001

— Rule —
PW-REL-001 • Hard wait detected • CRITICAL

— Why it's a problem —

```

`page.waitForTimeout(2000)` pauses for a fixed 2 seconds regardless of whether the element is ready. On a slow CI machine this causes false failures; on a fast local machine it hides timing bugs entirely.

— Suggested fix —

```
- await page.waitForTimeout(2000);
- await page.click('#submit-btn');
+ await page.locator('#submit-btn').click();
// Playwright auto-waits for the element to be ready before clicking.

✓ Done • copy the fix above and replace the matching lines in your file
```

AGENT 4 ✨ Rule suggestion agent — discover & add new rules

AI reads a sample of your actual codebase and suggests custom rules specific to your patterns — things the built-in rules don't cover. Each suggestion includes the rule title, description, pattern to detect, and recommended fix.

a Generate rule suggestions from your codebase

```
qcbot suggest-rules ./functional-tests --provider anthropic
```

→ Samples up to 8 files (1500 chars each), returns up to 6 rule suggestions

b Save suggestions to a file, review & edit

```
qcbot suggest-rules ./functional-tests --provider anthropic --output my-rules.json
```

→ Writes rule definitions as JSON — edit titles, severities, or patterns before using

c Or write a rule manually in JSON

Create `custom-rules.json` with your own rules:

```
[
  {
    "id": "TEAM-001",
    "title": "Prefer data-testid over CSS class selectors",
    "severity": "warning",
    "pattern": "\\\\.locator\\('\\\\\\.css-",
    "message": "Use getByTestId() – resilient to style changes"
  }
]
```

d Apply your custom rules to any scan

```
qcbot check ./tests --custom-rules ./my-rules.json
```

→ Custom rules run alongside all 59 predefined rules — findings appear in the same report

e

Or use the browser UI

Rules tab → ✨ **Suggest rules from codebase** → each card has an **Add rule** button. Click **Export JSON** to download your full rule set (including custom rules) for use in CI with `--custom-rules`.



WHAT YOU SEE IN THE TERMINAL

```
$ qcbot suggest-rules ./functional-tests --provider anthropic
⚡ qcBot rule suggestion agent · provider: anthropic

Sampling files from ./functional-tests ...
Selected 8 files · 1500 chars each · sending to AI ...

— Suggested rules —

Rule 1: Prefer data-testid selectors over CSS class selectors
Pattern: page.locator('.css-') or locator('[class*=')
Fix: use page.getByTestId('...') — resilient to style changes
Severity: warning

Rule 2: waitForResponse used without timeout — can hang indefinitely
Pattern: page.waitForResponse('/api/') without { timeout } option
Fix: add { timeout: 10_000 } as second argument
Severity: warning

Rule 3: Duplicate test description across describe blocks
Pattern: test('should login') appears 3+ times in this codebase
Fix: make test names unique and descriptive
Severity: info

... 3 more suggestions ...

✓ 6 rules suggested · run with --output my-rules.json to save
Then: qcbot check ./tests --custom-rules ./my-rules.json
```

Workflow tip: Run `qcbot suggest-rules --output my-rules.json` → review the file → commit it to your repo → reference it in CI with `--custom-rules ./my-rules.json`. Your team shares the same custom rule set automatically.



Removing custom rules

CLI — edit the JSON file and re-run

Open `my-rules.json`, delete the rule object you want to remove, save, then run:

```
# Remove TEAM-001 from the file, then verify it's gone:
qcbot check ./tests --custom-rules ./my-rules.json
# TEAM-001 findings will no longer appear
```

CLI — remove a specific rule by ID in one line

```
# Remove TEAM-001 from my-rules.json using node:
node -e "const fs=require('fs'); const r=JSON.parse(fs.readFileSync('my-rules.json')); fs.writeFileSync('my-rules.json', JSON.stringify(r.filter(x=>x.id!=='TEAM-001'),null,2));"

# Or disable a rule without deleting it — add "disabled": true:
# { "id": "TEAM-001", "disabled": true, ... }
```

Browser UI — remove a custom rule from the Rules tab

Rules tab → **Custom rules** section → click the  **delete** icon on any custom rule card → rule is removed immediately from the session and from localStorage. Click **Export JSON** afterwards to save the updated set for CI.

API key setup: Set one of these environment variables before running any agentic command — see [Section 2 → Step 5](#) for the full walkthrough.

```
export ANTHROPIC_API_KEY=sk-ant-... (Anthropic Claude — recommended)
```

```
export GOOGLE_API_KEY=AIza... (Google Gemini)
```

Or pass `--api-key YOUR_KEY --provider anthropic` directly on the command.

7

COMPLETE RULES REFERENCE (59 PREDEFINED RULES)

All rules run automatically via Engine 1. Severities: **critical** **warning** **info**

Selectors (PW-SEL) — 6 rules

Rule ID	Sev	What it catches
PW-SEL-001	CRIT	XPath locators (xpath= / locator('//...'))
PW-SEL-002	INFO	CSS ID selectors — locator('#...')
PW-SEL-003	INFO	No user-facing locators (getByRole/Label/Text/TestId) in spec file
PW-SEL-004	INFO	Index-based locator (.nth) — breaks on DOM reorder
PW-SEL-005	INFO	Legacy string selector engine (text= / css=)
PW-SEL-006	WARN	Same XPath string repeated 2+ times in one file

Reliability (PW-REL) — 8 rules

Rule ID	Sev	What it catches
PW-REL-001	CRIT	Hard waits — page.waitForTimeout()
PW-REL-002	WARN	page.\$\$() without web-first assertion — race condition risk
PW-REL-003	WARN	waitForSelector used — prefer auto-waiting locators
PW-REL-004	WARN	networkidle wait — flaky on polling connections
PW-REL-005	WARN	Deprecated waitForNavigation()
PW-REL-006	WARN	Conditional on element state — if (await ...isVisible())
PW-REL-007	WARN	force: true bypasses actionability checks
PW-REL-008	WARN	Low-level DOM access — \$eval / \$\$eval / elementHandle

Assertions (PW-AST) — 5 rules

Rule ID	Sev	What it catches
PW-AST-001	WARN	expect(await ...) anti-pattern — no assertion retries
PW-AST-002	CRIT	No expect() assertions in test — test passes without checks
PW-AST-003	INFO	Weak assertion — toBeTruthy/toBeFalsy instead of concrete state

Rule ID	Sev	What it catches
PW-AST-004	WARN	Non-Playwright assertion library (Node assert / chai) — no retries

PW-AST-005 INFO Brittle snapshot assertion — toMatchSnapshot on text/DOM

Structure (PW-STR) — 3 rules

Rule ID	Sev	What it catches
PW-STR-001	INFO	No describe() grouping — hard to navigate large suites
PW-STR-002	WARN	describe.serial / mode:'serial' — couples tests, no parallelism
PW-STR-003	INFO	beforeAll/afterAll shared state — order-dependent flakiness

Security (PW-SEC) — 4 rules

Rule ID	Sev	What it catches
PW-SEC-001	CRIT	Hardcoded secret — password/token/API key literal
PW-SEC-002	WARN	ignoreHTTPSErrors: true — TLS validation disabled
PW-SEC-003	CRIT	Credentials embedded in URL — user:password@host
PW-SEC-004	WARN	--disable-web-security launch flag

Coding Standards (PW-STD + PW-NMC + PW-CPX) — 20 rules

Rule ID	Sev	What it catches
PW-STD-001	CRIT	Focused test — test.only / describe.only committed to repo
PW-STD-002	WARN	page.pause() left in code — hangs CI
PW-STD-003	INFO	console.log/debug left in tests — noisy reports
PW-STD-004	INFO	Skipped / fixme test committed — test.skip / test.fixme
PW-STD-005	INFO	Arbitrary sleep — setTimeout / custom sleep()
PW-STD-006	WARN	Unfinished / AI-boilerplate code — TODO stubs, YOUR_VALUE_HERE
PW-NMC-001	WARN	Variable name should be camelCase NEW
PW-NMC-002	CRIT	Class name should be PascalCase NEW
PW-NMC-003	INFO	Helper function should be camelCase in spec file NEW

Rule ID	Sev	What it catches
PW-NMC-004	INFO	Non-descriptive test name (too short or generic) NEW
PW-NMC-005	WARN	Interface name should be PascalCase NEW
PW-NMC-006	WARN	Type alias should be PascalCase NEW
PW-NMC-007	WARN	Enum name should be PascalCase NEW
PW-NMC-008	INFO	Boolean variable missing is/has/can prefix NEW
PW-NMC-009	INFO	describe() block name starts with lowercase NEW
PW-NMC-010	INFO	Underscore-prefixed variable (outdated private notation) NEW
PW-CPX-001	WARN	Cyclomatic complexity > 12 branch points per file NEW
PW-CPX-002	INFO	Deeply nested code — 4+ levels deep NEW
PW-CPX-003	INFO	Spec file too long — > 200 lines with > 80 avg lines per test NEW
PW-CPX-004	INFO	Function with 5+ parameters NEW

**CI Config (PW-CI) — 3 rules**

ID	What it catches
PW-CI-001	retries: 0 in playwright.config — no retry on flake
PW-CI-002	workers: 1 — disables parallelism
PW-CI-003	timeout set below 5 000 ms — too tight for CI

**Mobile & Cross-env (PW-MOB) — 2 rules**

ID	What it catches
PW-MOB-001	Hard-coded viewport size — breaks responsive tests
PW-MOB-002	No mobile project in config but mobile pages tested

**Performance (PW-PER) — 3 rules**

ID	What it catches
PW-PER-001	No parallelism config — slow large suites
PW-PER-002	All tests in one file — no parallel sharding possible
PW-PER-003	page.reload() inside test loop — expensive anti-pattern

**Accessibility (PW-A11Y) — 2 rules**

ID	What it catches
PW-A11Y-001	No axe / accessibility assertion in suite
PW-A11Y-002	getText used for interactive element — prefer getByRole

59 total predefined rules — all run automatically every scan. Toggle any rule off in the Rules tab. Add your own custom rules via ✨ **Suggest rules** (Engine 2) or write them manually — see [Section 6 → Agent 4](#). Disable rules project-wide with `--disabled-rules PW-REL-001,PW-NMC-010`.

8 SUPPORTED STACKS

Use `qcbot stacks` to list all available options. Pass the stack id with `--stack <id>`.

Stack ID	Language / Framework	Example command
<code>playwright</code>	Playwright TypeScript (default)	<code>qcbot check ./tests --stack playwright</code>
<code>typescript</code>	TypeScript (generic)	<code>qcbot check ./src --stack typescript</code>
<code>pytest_api</code>	Python / pytest API tests	<code>qcbot check ./tests --stack pytest_api</code>
<code>playwright_python</code>	Playwright Python	<code>qcbot check ./tests --stack playwright_python</code>
<code>restassured</code>	Java / REST-assured	<code>qcbot check ./src/test --stack restassured</code>
<code>karate</code>	Karate DSL	<code>qcbot check ./features --stack karate</code>
<code>postman</code>	Postman / Newman	<code>qcbot check ./collections --stack postman</code>
<code>java_api</code>	Java API tests (generic)	<code>qcbot check ./src/test --stack java_api</code>
<code>playwright_java</code>	Playwright Java	<code>qcbot check ./tests --stack playwright_java</code>
<code>python_api</code>	Python API tests (generic)	<code>qcbot check ./tests --stack python_api</code>
<code>ts_frontend</code>	TypeScript frontend tests	<code>qcbot check ./tests --stack ts_frontend</code>
<code>python_frontend</code>	Python frontend tests	<code>qcbot check ./tests --stack python_frontend</code>

9 TROUBLESHOOTING

Problem	Likely cause	Fix
<code>qcbot: command not found</code>	Global install missing or PATH not updated	Run <code>npm install -g @cqs/qcbot</code> and open a new terminal
Score is 0 / no findings	Wrong <code>--stack</code> flag for your language	Run <code>qcbot stacks</code> and pick the matching id
AI command fails immediately	API key not set or wrong env var name	Run <code>echo \$ANTHROPIC_API_KEY</code> — if empty, see Section 2 → Step 5
AI command returns garbled output	Model hallucination or context too large	Try <code>--model claude-haiku-4-5-20251001</code> for faster/cheaper output
Browser UI won't load files	Browser blocked folder access permission	Click Allow in the browser permission dialog that appears
🌟 buttons not visible in Browser UI	AI not enabled in Settings	Open ⚙ Settings → check <i>Use AI for new scans</i> → paste key → Done
Python: <code>node not found</code>	Node.js not on PATH	Install Node.js 18+ from nodejs.org then reopen terminal
Custom rules not applied	Wrong path to JSON file	Use absolute path: <code>qcbot check ./tests --custom-rules /abs/path/rules.json</code>